

Intro To Operating Systems

2026 Semester 2 COMPSCI 340: Operating Systems
Talia Xu

Lecture 1
2.0.1

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

About Me: My name is Talia



Lecturer

Computer Science

Research Interest:

Embedded Systems,
Low Power Wireless Communication,
IoT / Batteryless Systems.

I mostly teach **systems courses**,
operating systems, computer organization, etc.

Agenda

1. Course logistics and assessments
2. Learning OS with xv6, C, and Rust
3. Why OS design choices matter

Lecture Times and Location

Tues:	11am - 12pm	401-439
Wed:	10am - 11am	109-B15
Fri:	10am - 11am	303-G23

Lecture Slides: Generally available by the end of the day before each lecture.

Lecture Recordings: Posted by the end of the day of the lecture.

Practice Questions: Lectures will include practice questions similar to those you will see on the test and final exam.

Interactive Quizzes: Any interactive quizzes used in class will not be posted separately, but you can review them in the lecture recordings.

Your Instructors

Talia Xu

(Course Director/Coordinator)

Instructor: Week 1 - 6

Office: 303S-592

Email: talia.xu@auckland.ac.nz

Office hour: by appointment

Sean Ma

Instructor: Week 7 - 12

Office: 303S-587

Email: sean.ma@auckland.ac.nz

Office hour: by appointment

Tutorial Times and Location

Wed: 2pm - 3pm 104-G63

Thur: 12pm - 1pm 303-101

Format: Tutorials are a mix of reviews of course content, practice questions, and help sessions for assignments.

Resources: Tutorials are not recorded, but slides will be shared before each tutorial.

Attendance: Tutorials are not compulsory, but are strongly recommended. Up to 1 bonus mark will be given to the practical component of the course if you attend at least 8 out of the 10 tutorials over the semester.

Tutors

Connor Williams

Email: cwil915@aucklanduni.ac.nz

Jinghao Jiang

Email: jjia496@aucklanduni.ac.nz

Class Reps

Any volunteers?

Operating Systems Is About Design Choices

Lectures develop the concepts and design rationale.

Assignments and tutorials apply them to an actual OS.

- What problem does a design solve?
- What trade-offs does it make?
- When would a different design be better?

Understanding the rationale matters more than memorising names.

Evaluation for this course

Assessment	Weight	Due Date
Assignment 1	10%	Aug 21
Midterm	20%	Aug 27 @ 6:30 pm
Assignment 2	10%	Sep 25
Assignment 3	10%	Oct 23
Assignment 4 (Optional)	10%	TBD
Final	50%	TBD

Assignments

Throughout the course, we will work with xv6, a teaching operating system.

Each assignment asks you to implement or examine one aspect of an operating system.

Assignments are offered in two languages: C and Rust.

You can choose one of the two languages for each assignment.

xv6: A Small Unix-like OS

xv6 is a teaching operating system.

- Small enough to read, real enough to modify
- Shows processes, system calls, traps, scheduling, and filesystems
- Assignments ask you to change kernel behaviour directly

C and Rust in Systems

C

- Traditional OS language
- Direct pointer and memory control
- Many safety checks are manual

Rust

- Systems control with stronger checks
- Ownership makes mutation explicit
- Helps prevent memory and data-race bugs

Both require understanding memory, concurrency, and hardware boundaries.

Why Rust Is Worth Learning

- Companies and open-source communities increasingly use Rust in real systems
- Android uses Rust for native OS components; the Linux kernel supports Rust development
- Memory safety without garbage collection suits security- and performance-critical code
- Ownership and borrowing make assumptions explicit; these skills transfer to C and OS design

Same Idea, Different Contract

C

```
void bump_all(int *xs, size_t len) {  
    for (size_t i = 0; i < len; i++) {  
        xs[i] += 1;  
    }  
}
```

Pointer + length: the caller must keep them correct.

Rust

```
fn bump_all(xs: &mut [i32]) {  
    for x in xs {  
        *x += 1;  
    }  
}
```

Mutable slice: length and exclusive access are in the type.

Assignment 4: Optional Project

Assignments are best 3 of 4.

- Assignment 4 is optional and can replace one assignment grade
- You may skip one of Assignments 1–3 and do Assignment 4 instead
- Explore an OS feature, extension, or mechanism that interests you
- Proposal: message Talia and Zalan (zvar509@aucklanduni.ac.nz) and have it approved by the end of Week 10
- Code due: second-last week of the exam period; exact deadline TBA
- Marking: code review and a short recorded demo

To pass this course

Overall Mark: To pass the course, you must achieve at least 50% for your overall course mark.

Test/Exam Component: You must also pass the combined test/exam component. This test/exam component is worth 70% of the overall mark, consisting of the test (worth 20%) and the final exam (worth 50%). To pass this component, you need to get at least 35 out of 70.

Midterm is in week 6. Details to come.

Aegrotat / Compassionate Consideration Policy

If circumstances impacted your ability to sit an invigilated assessment, or impaired your preparation/performance, we will organise an alternative assessment.

The alternative assessment may be in the same format or in an alternative format such as viva voce/oral test.

The Recommended Books Complement Lectures

“Operating Systems: Three Easy Pieces”

by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau

“Silberschatz’s Operating System Concepts, 10th Edition, Global Edition”

by Abraham Silberschatz and Peter B. Galvin and Greg Gagne

“The C Programming Language”

by Brian Kernighan and Dennis Ritchie

“The Rust Programming Language”

by Steve Klabnik, Carol Nichols, Chris Krycho, and contributors

Skills You Should Practice Again If Needed

Programming

- C Programming and Debugging
- Python Programming and Debugging

Basic Concepts

- Being able to convert between binary, hex, and decimal
- Memory being byte-addressable, memory addresses (pointers)

Why Operating Systems?

```
int array[ROWS][COLS];  
for (int i = 0; i < ROWS; i++) {  
    for (int j = 0; j < COLS; j++) {  
        array[i][j] = i * COLS + j;  
    }  
}
```

```
int array[ROWS][COLS];  
for (int j = 0; j < COLS; j++) {  
    for (int i = 0; i < ROWS; i++) {  
        array[i][j] = i * COLS + j;  
    }  
}
```

Why Operating Systems?

```
// 1 + 2 + 3 + 4 + 5 + ... + n = 100
```

```
int iterative_addition(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += i;  
    }  
    return sum;  
}
```

```
int recursive_addition(int n) {  
    if (n == 0) return 0;  
    return (n - 1) + recursive_addition(n - 1);  
}
```